Unit-IV: Queue

A **Queue** is defined as a linear data structure that is open at both ends and the operations are performed in **First In First Out** (FIFO) order.

❖ **Queue Representation:**

Like stacks, Queues can also be represented in an array: In this representation, the Queue is implemented using the array. Variables used in this case are

- **Queue:** the name of the array storing queue elements.
- **Front**: the index where the first element is stored in the array representing the queue.
- **Rear:** the index where the last element is stored in an array representing the queue.

Queue Data Structure

❖ **Types of Queues:**

There are **five different types of queues** that are used in different scenarios. They are:

1. Input Restricted Queue (this is a Simple Queue)
2. Output Restricted Queue (this is also a Simple Queue)
3. Circular Queue
4. Double Ended Queue (Deque)
5. Priority Queue
   - Ascending Priority Queue
   - Descending Priority Queue

1. Circular Queue: Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle. It is also called **'Ring Buffer'**. This queue is primarily used in the following cases:

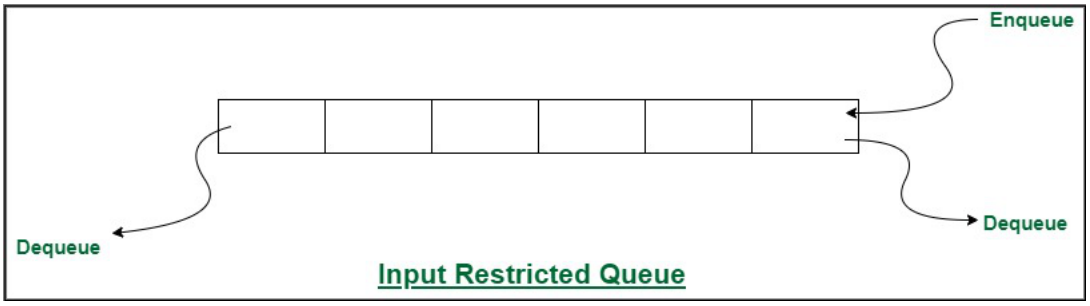
i. **Memory Management:** The unused memory locations in the case of ordinary queues can be utilized in circular queues.

ii. **Traffic system:** In a computer-controlled traffic system, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set.

iii. **CPU Scheduling:** Operating systems often maintain a queue of processes that are ready to execute or that are waiting for a particular event to occur.

The time complexity for the circular Queue is O(1).

2. Input Restricted Queue: In this type of Queue, the input can be taken from one side only(rear) and deletion of elements can be done from both sides (front and rear). This kind of Queue does not follow FIFO (first in first out). This queue is used in cases where the consumption of the data needs to be in FIFO order but if there is a need to remove the recently inserted data for some reason and one such case can be irrelevant data, performance issue, etc.



Input Restricted Queue

Advantages of Input Restricted Queue:

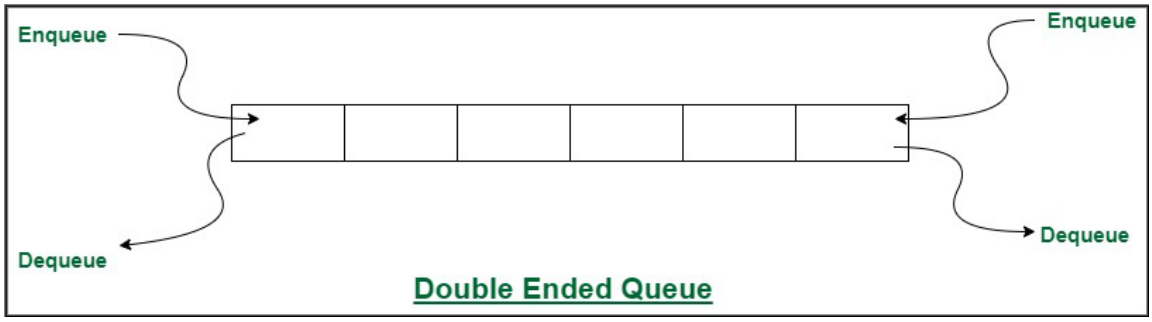
- Prevents overflow and overloading of the queue by limiting the number of items added.
- Helps maintain stability and predictable performance of the system.

Disadvantages of Input Restricted Queue:

- May lead to resource wastage if the restriction is set too low and items are frequently discarded.
- May lead to waiting or blocking if the restriction is set too high and the queue is full, preventing new items from being added.

3. Output restricted Queue: In this type of Queue, the input can be taken from both sides (rear and front) and the deletion of the element can be done from only one side(front). This queue is used in the case where the inputs have some priority order to be executed and the input can be placed even in the first place so that it is executed first.

4. Double ended Queue: Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (front and rear). That means, we can insert at both front and rear positions and can delete from both front and rear positions. Since Deque supports both stack and queue operations, it can be used as both. The Deque data structure supports clockwise and anticlockwise rotations in O(1) time which can be useful in certain applications. Also, the problems where elements need to be removed and or added both ends can be efficiently solved using Deque.



5. Priority Queue: A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. There are two types of Priority Queues. They are:

1. **Ascending Priority Queue:** Element can be inserted arbitrarily but only smallest element can be removed. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 2, 4, 8.
2. **Descending priority Queue:** Element can be inserted arbitrarily but only the largest element can be removed first from the given Queue. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 8, 4, 2.

The time complexity of the Priority Queue is O(logn).

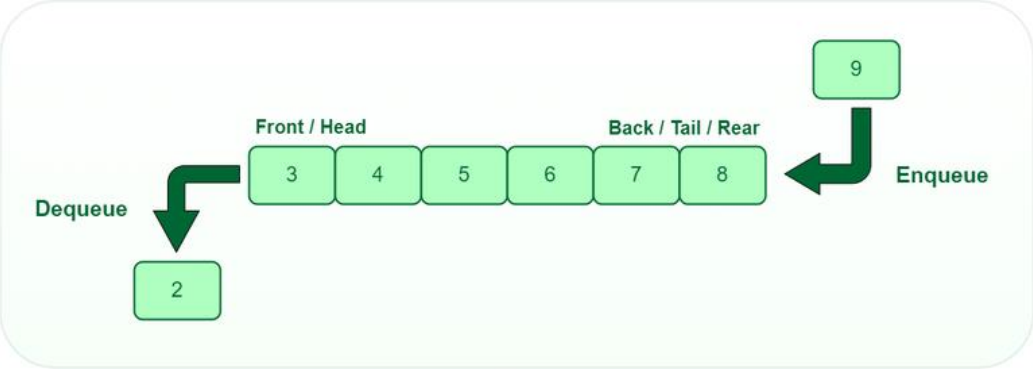
- **Issues of Queue:**

Some common issues that can arise when using queues:

1. Queue overflow: Queue overflow occurs when the queue reaches its maximum capacity and is unable to accept any more elements. This can cause data loss and can lead to application crashes.
2. Queue underflow: Queue underflow occurs when an attempt is made to remove an element from an empty queue. This can cause errors and application crashes.
3. Priority inversion: Priority inversion occurs in priority queues when a low-priority task holds a resource that a high-priority task needs. This can cause delays in processing and can impact system performance.
4. Deadlocks: Deadlocks occur when multiple threads or processes are waiting for each other to release resources, resulting in a situation where none of the threads can proceed. This can happen when using concurrent queues and can lead to system crashes.
5. Performance issues: Queue performance can be impacted by various factors, such as the size of the queue, the frequency of access, and the type of operations performed on the queue. Poor queue performance can lead to slower system performance and reduced user experience.
6. Synchronization issues: Synchronization issues can arise when multiple threads are accessing the same queue simultaneously. This can result in data corruption, race conditions, and other errors.
7. Memory management issues: Queues can use up significant amounts of memory, especially when processing large data sets. Memory leaks and other memory management issues can occur, leading to system crashes and other errors.

Basic Operations on Queue:
Some of the basic operations for Queue in Data Structure are:

- **enqueue() –** Insertion of elements to the queue.
- **dequeue() –** Removal of elements from the queue.
- **peek() or front()-** Acquires the data element available at the front node of the queue without deleting it.
- **rear() –** This operation returns the element at the rear end without removing it.
- **isFull() –** Validates if the queue is full.
- **isEmpty() –** Checks if the queue is empty.
- **size():** This operation returns the size of the queue i.e. the total number of elements it contains.

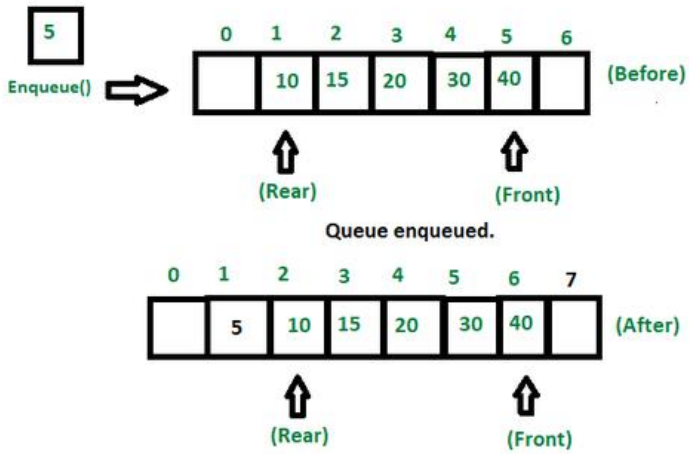


Queue Data Structure

Operation 1: enqueue()
Inserts an element at the end of the queue i.e. at the rear end.

The following steps should be taken to enqueue (insert) data into a queue:

- Check if the queue is full.
- If the queue is full, return overflow error and exit.
- If the queue is not full, increment the rear pointer to point to the next empty space.
- Add the data element to the queue location, where the rear is pointing.
- return success.

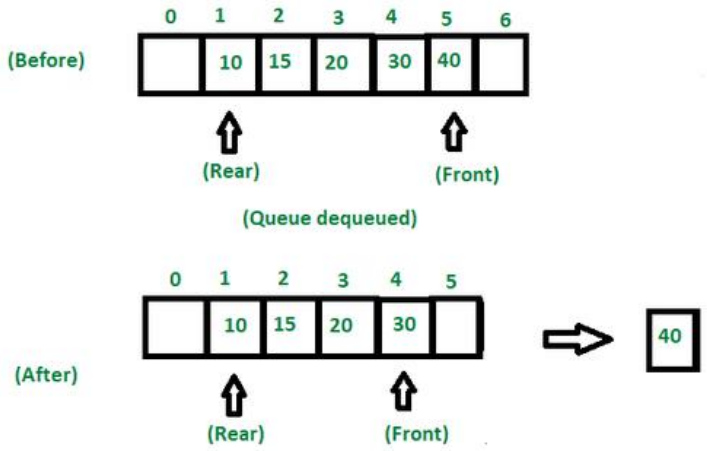


Enqueue representation

Operation 2: dequeue()
This operation removes and returns an element that is at the front end of the queue.

The following steps are taken to perform the dequeue operation:

- Check if the queue is empty.
- If the queue is empty, return the underflow error and exit.
- If the queue is not empty, access the data where the front is pointing.
- Increment the front pointer to point to the next available data element.
- The Return success.



Dequeue operation

Operation 3: front()
This operation returns the element at the front end without removing it.

The following steps are taken to perform the front operation:

- If the queue is empty return the most minimum value.
- otherwise, return the front value.

Operation 4: rear()

This operation returns the element at the rear end without removing it.

The following steps are taken to perform the rear operation:

- If the queue is empty return the most minimum value.
- otherwise, return the rear value.

Operation 5: isEmpty():
This operation returns a Boolean value that indicates whether the queue is empty or not.

The following steps are taken to perform the Empty operation:

- check if front value is equal to -1 or not, if yes then return true means queue is empty.
- Otherwise return false, means queue is not empty

Operation 6: isFull()
This operation returns a boolean value that indicates whether the queue is full or not.

The following steps are taken to perform the isFull() operation:

- Check if front value is equal to zero and rear is equal to the capacity of queue if yes then return true.
- otherwise return false.

Operation 7: size()

This operation returns the size of the queue i.e. the total number of elements it contains.

❖ **Applications of Queue data structure:**

1. **Task Scheduling**: Queues can be used to schedule tasks based on priority or the order in which they were received.
2. **Resource Allocation:** Queues can be used to manage and allocate resources, such as printers or CPU processing time.
3. **Batch Processing**: Queues can be used to handle batch processing jobs, such as data analysis or image rendering.
4. **Message Buffering**: Queues can be used to buffer messages in communication systems, such as message queues in messaging systems or buffers in computer networks.
5. **Event Handling**: Queues can be used to handle events in event-driven systems, such as GUI applications or simulation systems.
6. **Traffic Management**: Queues can be used to manage traffic flow in transportation systems, such as airport control systems or road networks.
7. **Operating systems:** Operating systems often use queues to manage processes and resources. For example, a process scheduler might use a queue to manage the order in which processes are executed.
8. **Network protocols:** Network protocols like TCP and UDP use queues to manage packets that are transmitted over the network. Queues can help to ensure that packets are delivered in the correct order and at the appropriate rate.
9. **Printer queues:** In printing systems, queues are used to manage the order in which print jobs are processed. Jobs are added to the queue as they are submitted, and the printer processes them in the order they were received.
10. **Web servers:** Web servers use queues to manage incoming requests from clients. Requests are added to the queue as they are received, and they are processed by the server in the order they were received.
11. **Breadth-first search algorithm:** The breadth-first search algorithm uses a queue to explore nodes in a graph level-by-level. The algorithm starts at a given node, adds its neighbours to the queue, and then processes each neighbour in turn.